

SIMATS ENGINEERING

ASSESSMENT TOOL 2

Course Name: Object Oriented Analysis and Design

Course Code: CSA1105

Course Outcome: CO2 — Design UML diagrams to represent system requirements and architecture (BL3)

Weightage: 60%

Questions Answered: 4 of 6 — Use Case, Class, Sequence, and Activity Diagrams | **Total Marks: 40**

Annexure A | Application Based Questions

Code of Conduct:

I _____ RASHMI NAURENE (192521357) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____ *Rashmi* _____

Q1. Use Case Diagram Analysis — Smart Pharmacy Management System [10 Marks]

A Smart Pharmacy Management System allows customers to search medicines, upload prescriptions, place medicine orders, and make online payments. Pharmacists verify prescriptions and dispense medicines, while administrators manage inventory and user accounts. The current system design does not clearly define actor responsibilities and interactions. Analyse the system requirements, identify all actors and missing use cases, and design an optimized Use Case Diagram showing appropriate relationships (include, extend, and generalization) to improve system functionality.

ANALYSIS OF PROBLEM CONTEXT

The scenario names three responsibilities — order/pay for the Customer, verify/dispense for the Pharmacist, and manage inventory/accounts for the Administrator — but does not mention how a user gets authenticated or how a completed order is billed. Both Pharmacist and Administrator are also, at their core, internal staff who log in the same way, so it makes sense to model them as specialisations of a common Staff actor rather than as three unrelated actors. These two observations shape the design below: authentication is pulled out as a shared included use case, and Pharmacist/Administrator inherit from Staff through generalization.

ACTORS IDENTIFIED

Actor	Role in the System
Customer	Primary actor who searches medicines, uploads prescriptions, places orders, and pays online.
Staff (abstract)	Represents any internal user who logs in through the same interface; specialised by Pharmacist and Administrator below.
Pharmacist	Specialises Staff — verifies uploaded prescriptions and dispenses medicine.
Administrator	Specialises Staff — manages medicine inventory and user accounts.

MISSING USE CASE IDENTIFIED

The original description stops at 'make online payments' without mentioning what happens afterward. A completed payment naturally needs a receipt, so Generate Invoice is added as an optional use case that extends Make Payment — this was the one clear gap in the given requirements.

RELATIONSHIPS DEFINED

Relationship	Justification
Place Order --include--> Login	Ordering is only possible for an authenticated customer, so login is a mandatory sub-flow.
Dispense Medicine --include--> Verify Prescription	Medicine can never be handed over without a verified prescription — mandatory, not optional.
Generate Invoice --extend--> Make Payment	An invoice is produced only if the customer chooses to download one after a successful payment — optional.
Pharmacist / Administrator --generalization--> Staff	Both roles are specialised kinds of Staff and inherit the same login behaviour while adding their own domain-specific use cases.

Optimized Use Case Diagram:

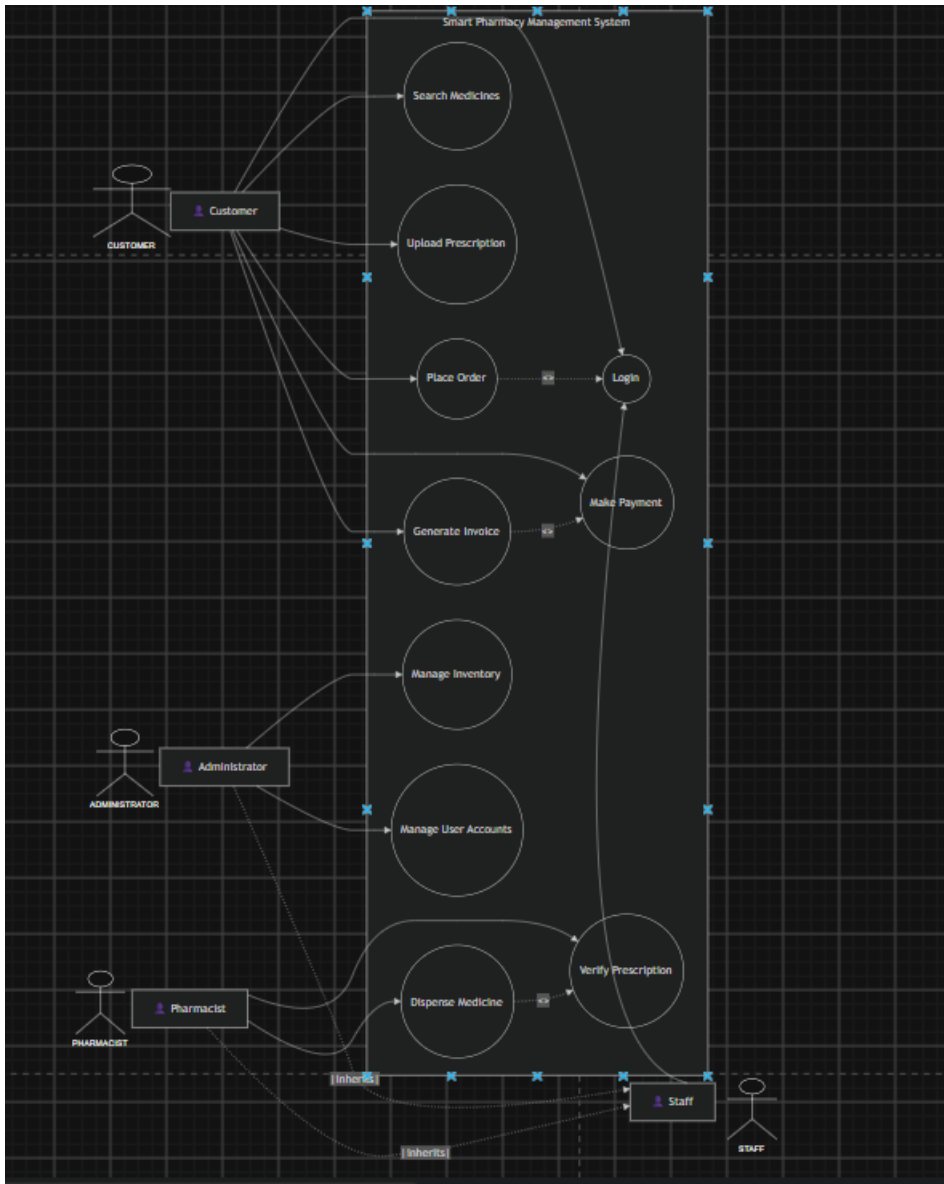


Figure 1.1 — Use Case Diagram for the Smart Pharmacy Management System

Customer associates with Login, Search Medicines, Upload Prescription, Place Order, and Make Payment. Staff is drawn as an abstract actor with Pharmacist and Administrator specialising it below, each connecting only to its own use cases (Verify Prescription and Dispense Medicine for the Pharmacist; Manage Inventory and Manage User Accounts for the Administrator), so no actor's association lines cross into another actor's territory. This closes the two gaps identified above — authentication is now an explicit mandatory step, and invoicing is available as an optional extension — while keeping the diagram easy to trace at a glance.

Q2. Class Diagram Design — Smart Library Management System [10 Marks]

A Smart Library Management System consists of entities such as Book, Member, Librarian, Reservation, Fine, and BorrowRecord. The existing system does not clearly specify class attributes, operations, or relationships among these entities. Analyze the system, identify relevant classes with appropriate attributes and methods, define relationships (association, aggregation, composition, and inheritance), specify multiplicity, and construct a well-structured Class Diagram.

ANALYSIS OF PROBLEM CONTEXT

The six named entities cover the core of a library workflow, but the question also asks for an inheritance relationship, which none of the six provide on their own. In practice, a library's Student members and Faculty members are charged with different borrowing limits and privileges even though they share the same registration and search behaviour — so Member is modelled as a base class specialised into StudentMember and FacultyMember, which reflects a real policy difference rather than an artificial split.

CLASSES, ATTRIBUTES AND METHODS

Class	Attributes / Methods
Member (base class)	memberId, name searchBook(), reserveBook()
StudentMember	borrowLimit checkLimit() — inherits Member
FacultyMember	department checkLimit() — inherits Member
Book	bookId, title checkAvailability()
Librarian	librarianId issueBook()
Reservation	reservationId, status createReservation()
BorrowRecord	issueDate, dueDate returnBook(), checkOverdue()
Fine	amount payFine()

RELATIONSHIPS AND MULTIPLICITY

Relationship (with Multiplicity)	Justification
StudentMember / FacultyMember -- inheritance--> Member	Both are specialised kinds of Member that inherit its registration and search behaviour.
Member --association--> Reservation (1 to 0..*)	A member may place zero or many reservations; each reservation is made by exactly one member.
Reservation --association--> Book (1 to 1)	Each reservation is placed against exactly one book.
Book --association--> BorrowRecord (1 to 0..*)	A book accumulates a borrowing history over time.
Librarian --association--> BorrowRecord (1 to 0..*)	A librarian issues and processes many borrow records.
BorrowRecord --composition--> Fine (1 to 0..1)	A fine is generated only if a return is overdue and cannot exist independently of the borrow record that produced it — deleting the record removes its fine.

Class Diagram:

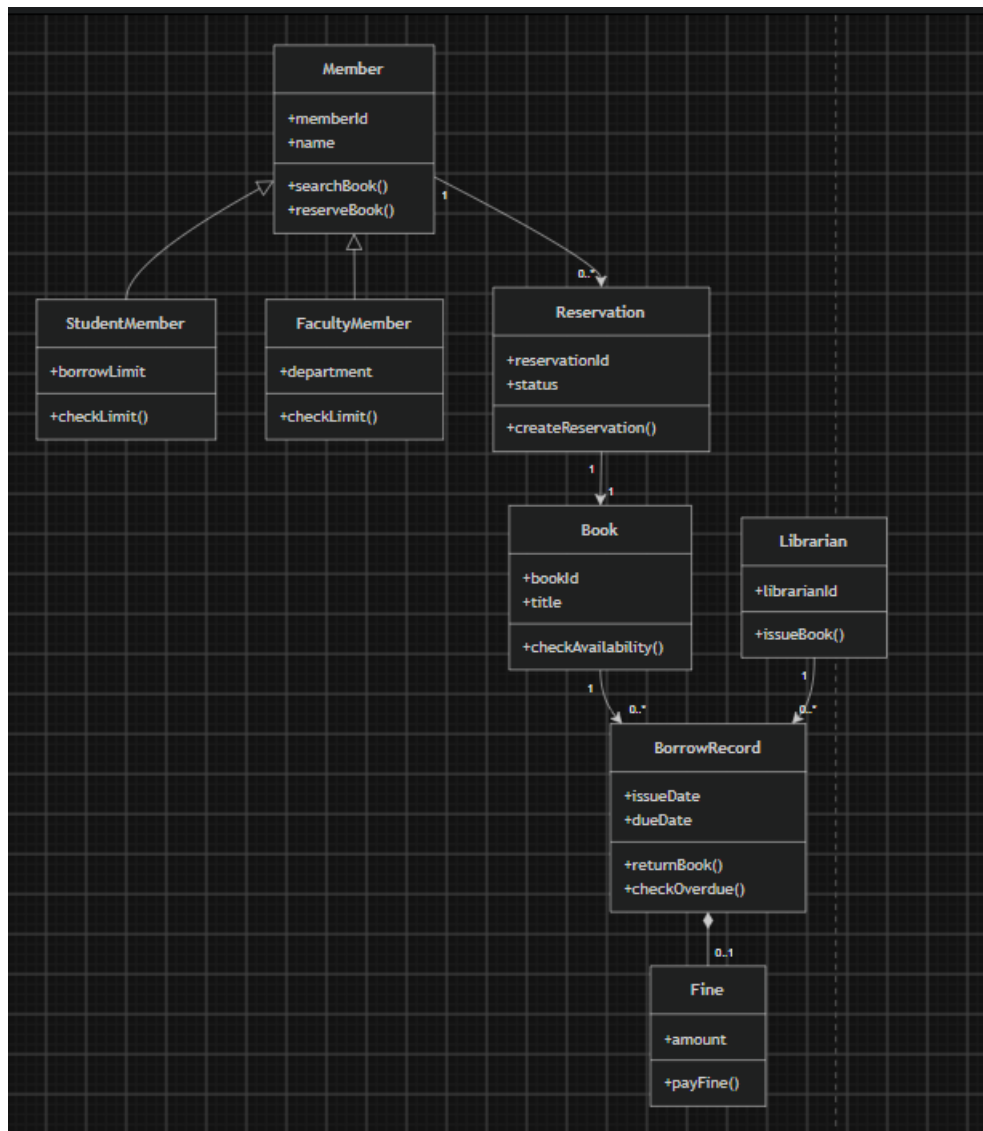


Figure 2.1 — Class Diagram for the Smart Library Management System

Member sits at the top with StudentMember and FacultyMember inheriting via hollow-triangle arrows. Book, Reservation, Librarian, and BorrowRecord form the operational core, and Fine is composed inside BorrowRecord (filled diamond, 0..1) since a fine only exists in the context of an overdue borrow record. Every relationship carries an explicit multiplicity so the diagram fully answers the question's requirements in one uncluttered view.

Q3. Sequence Diagram Evaluation — Online Examination System [10 Marks]

An Online Examination System enables students to log in, select an examination, answer questions, submit responses, and receive results. The communication among the student, examination module, evaluation module, and database is incomplete. Analyze the interaction process, identify all participating objects and missing message exchanges, and design a detailed Sequence Diagram representing the complete interaction flow with activation bars and lifelines.

ANALYSIS OF PROBLEM CONTEXT

The scenario says the student 'logs in' but does not name the object that performs authentication, so a LoginModule is added as the missing participant. It also stops at 'receive results' without describing how the score is produced — this is modelled by having ExamController hand the submitted responses to the Database in a single evaluateAndStore() call, which both scores the exam against the stored answer key and saves the result, keeping the interaction realistic without introducing an extra object purely for that one step.

PARTICIPATING OBJECTS IDENTIFIED

Object	Role
Student (actor)	The human actor who logs in, selects an exam, answers questions, and submits.
: LoginModule	Boundary object that authenticates the student's credentials against the database.
: ExamController	Control object that drives exam selection, question retrieval, and submission handling.
: Database	Stores credentials and question banks, and evaluates/stores the final result.

MESSAGE FLOW (AS MODELLLED IN THE DIAGRAM)

- 1: login() → 2: validateCredentials() → 3: return isValid → 4: loginSuccess()
- 5: selectExamination() → 6: getQuestions() → 7: return questionList → 8: showQuestions()
- 9: submitExam() → 10: evaluateAndStore(responses) → 11: return score → 12: displayResult(score)

Sequence Diagram:

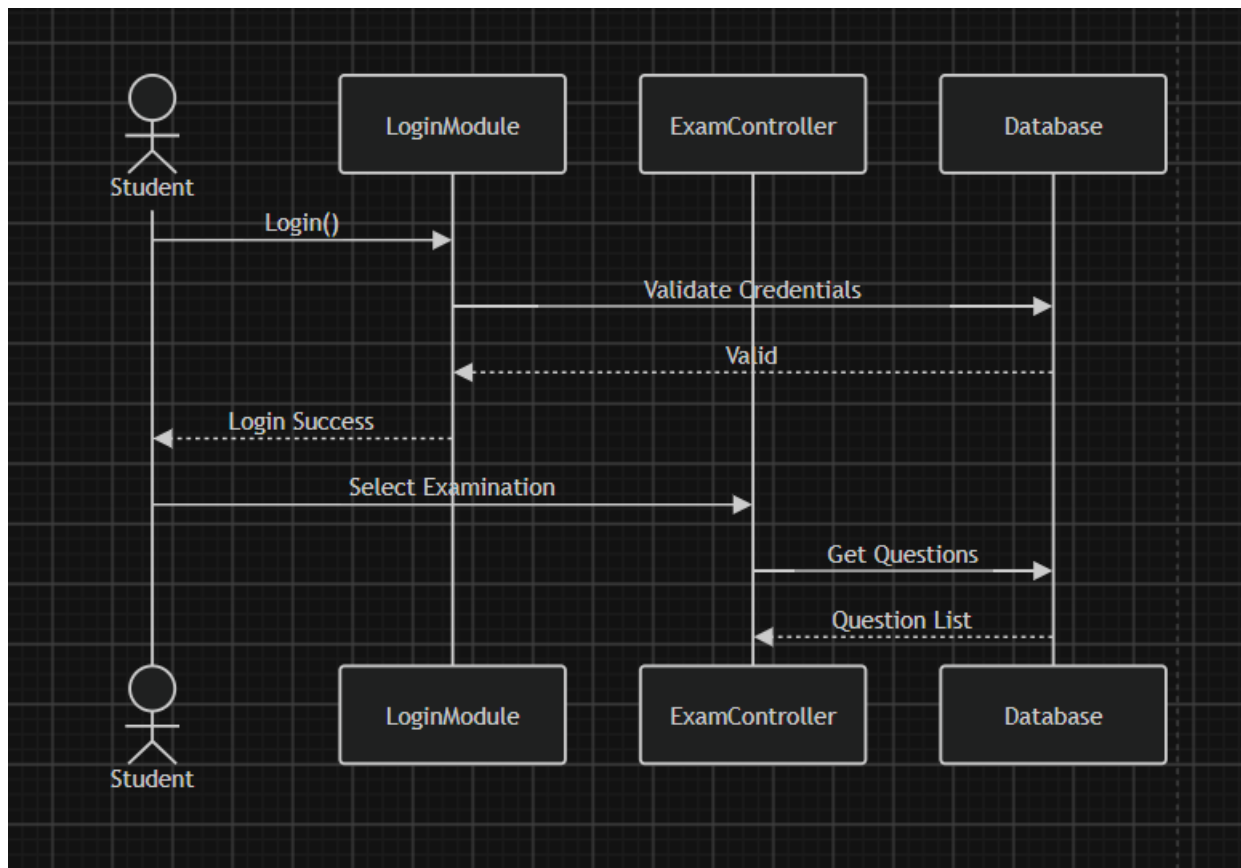


Figure 3.1 — Sequence Diagram for the Online Examination System

Each of the four lifelines carries an activation bar only for the period it is actively processing a request: LoginModule activates once, briefly, to authenticate; ExamController stays active across the exam-taking period since it mediates every later call; and Database activates three separate times — for login validation, for fetching questions, and again when the final result is evaluated and stored. This makes the object responsible for each step immediately clear from the diagram alone.

Q4. Activity Diagram Construction — Loan Approval System [10 Marks]

A Loan Approval System allows customers to submit loan applications, upload supporting documents, complete eligibility verification, receive loan approval or rejection, and complete loan disbursement. The workflow lacks proper decision points and parallel activities. Analyze the business process, identify all workflow activities, include decision nodes for approval and rejection, represent parallel processing where applicable, and construct an optimized Activity Diagram.

ANALYSIS OF PROBLEM CONTEXT

The scenario mentions 'eligibility verification' as a single step, but eligibility is realistically decided by two independent checks — creditworthiness and income/employment status — that don't depend on each other and can be assessed at the same time. Modelling them sequentially, as the original description implies, wastes processing time; splitting them into a fork/join pair is the optimisation the question asks for. The scenario also never mentions what happens when uploaded documents are invalid, so a validation decision with a resubmission loop is added before eligibility verification begins.

WORKFLOW ACTIVITIES IDENTIFIED

- Submit Loan Application and Upload Supporting Documents (Customer)
- Documents Valid? — decision node, looping back to document upload on [No]
- Credit Score Verification and Income & Employment Verification — run in parallel via fork/join
- Evaluate Eligibility, then Approved? — decision node branching to rejection or approval
- Approve Loan → Complete Loan Disbursement (happy path)

DECISION NODES AND PARALLEL PROCESSING

- **Decision 1 — Documents Valid?** an [No] branch loops back to document upload rather than terminating the process, since a customer with a rejected document should be able to resubmit.
- **Fork / Join** — Credit Score Verification and Income & Employment Verification run concurrently once documents are validated, shortening total turnaround.
- **Decision 2 — Approved?** [No] sends a rejection letter and ends the process; [Yes] proceeds to Approve Loan and Loan Disbursement.

Optimized Activity Diagram:

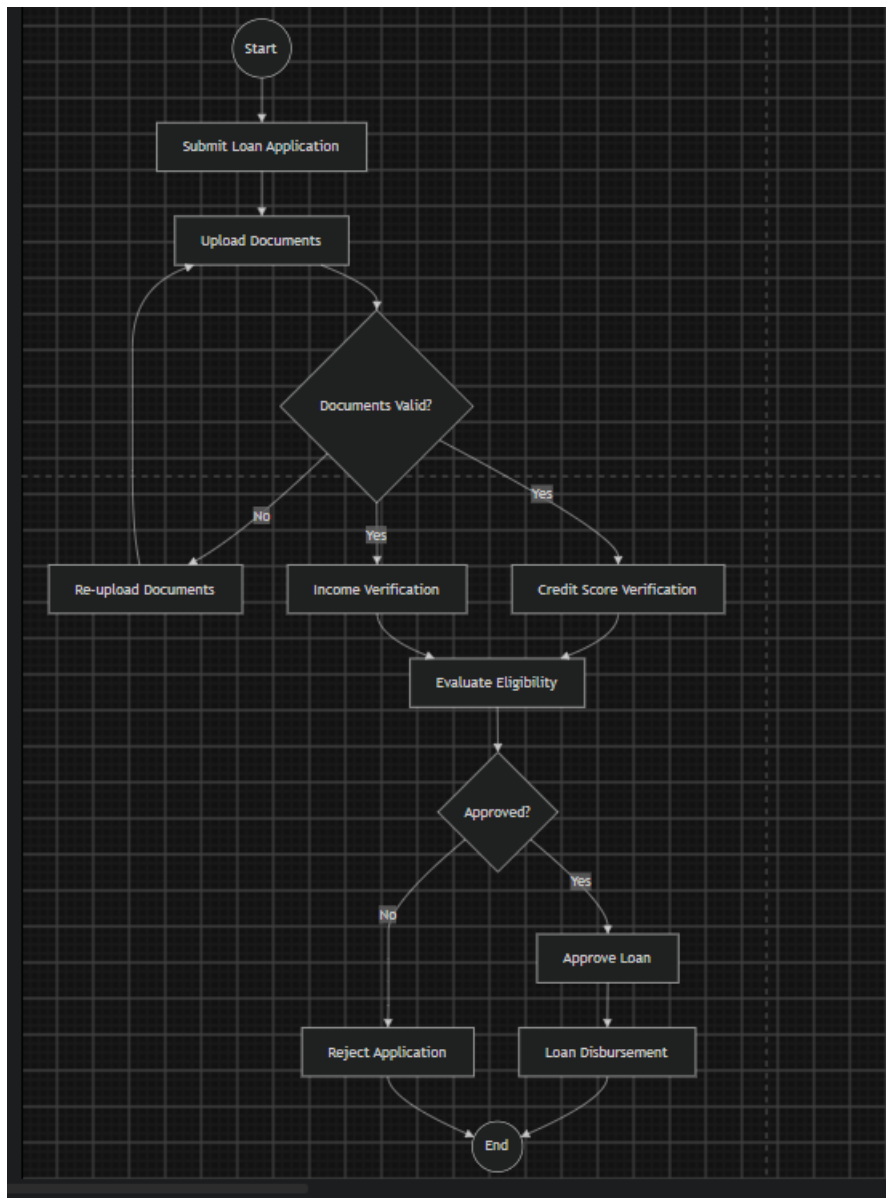


Figure 4.1 — Activity Diagram for the Loan Approval System

The diagram runs as a single main flow down the centre: the initial node leads through application submission and document upload into the Documents Valid? decision, whose [No] branch loops back on the left. Once documents are valid, the flow forks into the two independent eligibility checks, joins back together, and proceeds through Evaluate Eligibility and the Approved? decision, where [No] exits to a short rejection box on the right while [Yes] continues straight down through Approve Loan and Loan Disbursement to the final node.